WELT 2 · DOPPELSTUNDEN 15–16 · SORTIER-DUELL · IDE-LEVEL 2.3

W2·3 – Die Rangliste: Effizienz und binäre Suche

Dateien: `AQ_W2_SortierDuell_Geruest.txt` (DS 15) · `AQ_W2_L2_Rangliste_Geruest.txt` (IDE-Level 2.3, DS 16) · Sortier-Duell: Team-Wettbewerb · Im Lab: **Level 2.3 bis 2.5**

Niveau (KC 2017): Teil B (binäre Suche) Grund- und Leistungskurs · Teil A, Aufgabe 6 und Teil C (Effizienz: Zahl der Operationen und Speicherbedarf) nur Leistungskurs (eA)

Euer Spiel geht online. Aus der Highscore-Liste mit zehn Einträgen wird eine weltweite Bestenliste mit Hunderttausenden. Alle drei Sortierverfahren aus W2·2 sortieren korrekt – aber sind sie auch **schnell genug**? Und wie findet man in einer Million Einträgen in Sekundenbruchteilen den richtigen?

Heute misst und rechnest du, wie viel **Arbeit** ein Algorithmus macht – genau so, wie es im Abitur verlangt wird. Im **Sortier-Duell** treten die Teams gegeneinander an. Danach lernst du mit der **binären Suche** ein Verfahren kennen, das bei doppelt so vielen Daten nur einen einzigen Schritt mehr braucht.

Teil A – Effizienz und Sortier-Duell (DS 15) ◆ nur LK (eA)

Aufgabe 0: Zähl-Warm-up

PAPIER ★

Ohne Gerät: Die Methode `insertionSort()` der Rangliste (W2·2, absteigend, jeder Aufruf von besser zählt) sortiert drei Ranglisten mit je vier Einträgen. Gib die Zählerstände am Ende an.

Punkte (Index 0 bis 3)	Vergleiche	Vertauschungen	Vergleiche bei selectionSort
40, 30, 20, 10			
10, 20, 30, 40			
40, 10, 30, 20			

Infoteil 1: Wie misst man den Aufwand eines Algorithmus?

- **Nicht in Sekunden.** Die Laufzeit hängt vom Rechner, vom Browser und vom Tempo-Regler der Online-IDE ab – das wäre keine Aussage über den Algorithmus.
- **Sondern in Grundoperationen.** Beim Sortieren zählt man **Vergleiche** und **Vertauschungen** – genau das tun `besser`, `vertausche` und die Duell-Befehle `Vis.kleiner`, `Vis.vertausche`. Diese Zahl ist eine Eigenschaft des Algorithmus.
- **Und immer abhängig von n**, der Anzahl der Elemente. Spannend ist nicht „wie viele bei 10 Elementen“, sondern: **Wie wächst die Zahl, wenn n wächst?**
- Derselbe Algorithmus kann je nach Eingabe unterschiedlich viel Arbeit haben. Man unterscheidet drei Fälle:

Fall	Bedeutung	Beim absteigenden Sortieren der Rangliste
bester Fall	günstigste Eingabe der Länge n	schon absteigend sortiert
schlechtester Fall	ungünstigste Eingabe der Länge n	genau umgekehrt, also **aufsteigend** sortiert
mittlerer Fall	durchschnittliche Eingabe	zufällig gemischt

Im Abitur interessiert meist der **schlechteste Fall** – er ist eine Garantie: Schlimmer wird es nie.

Aufgabe 1: Messreihe mit der Visualisierung

IDE ★★

a) Lege im Workspace **AlgoQuest_W2** eine neue Datei mit dem Inhalt von `AQ_W2_SortierDuell_Geruest.txt` an und lies die ANLEITUNG. Starte das Programm: 40 Balken werden mit dem fertigen `bubbleSort` sortiert. Stelle das Tempo langsam und beschreibe in einem Satz das typische Bewegungsmuster.

b) **Training 1 und 2:** Programmiere in `MeinSortierer` die Methoden `selectionSort` und `insertionSort`. Achtung: Hier wird **aufsteigend** sortiert (kleinster Balken links) und du arbeitest nur mit `Vis.kleiner(i, j)` und `Vis.vertausche(i, j)`. `Vis.fertig()` färbt grün, wenn alles stimmt. Beschreibe auch hier das Bewegungsmuster.

c) **Messreihe** (Tempo auf Maximum, z. B. `SystemTools.setSpeed(10000);`): Starte mit `Vis.starten(n, "zufall")` bzw. `Vis.starten(n, "umgekehrt")` und trage die Zahl der **Vergleiche** ein. Die letzte Spalte rechnest du aus.

n	Bubblesort (Vorlage) zufall	Selectionsort zufall	Insertionsort zufall	Insertionsort umgekehrt	n(n − 1)/2
10					
20					
40					
80					

d) Was passiert mit der Zahl der Vergleiche, wenn n verdoppelt wird? Berechne für Insertionsort (zufall) die Faktoren 20 → 40 und 40 → 80 und formuliere eine Regel.

e) **Training 3** (n = 40, genau die Daten der Duell-Runden): Miss die Vergleiche aller drei Verfahren auf den vier Duell-Ausgangslagen.

Verfahren	zufall	fast sortiert	umgekehrt	viele gleiche	Summe
Bubblesort (Vorlage)					
Selectionsort					
Insertionsort					

f) Ein Verfahren braucht überall gleich viele Vergleiche. Begründe anhand seines Aufbaus, warum das so sein muss. Erläutere, warum dein `bubbleSort` aus Level 2.2 auf schon sortierten Daten deutlich weniger braucht als die Vorlage hier.

Aufgabe 2: Nachrechnen statt messen

PAPIER ★★

a) Selectionsort mit n Einträgen: Gib an, wie viele Vergleiche im 1., im 2. und im letzten Durchlauf der äußeren Schleife anfallen. Zeige, dass die Gesamtzahl **n(n − 1)/2** beträgt. **[3 BE]**

Tipp: Schreibe die Summe zweimal untereinander – einmal vorwärts, einmal rückwärts – und addiere spaltenweise (der junge Gauß lässt grüßen).

b) Berechne n(n − 1)/2 für n = 40, n = 80 und n = 1000 und vergleiche mit deiner Messreihe. **[1 BE]**

c) Zeige rechnerisch: Verdoppelt man n, so wächst die Zahl der Vergleiche ungefähr auf das **Vierfache**. Formuliere das Ergebnis in einem Satz, wie er im Abitur stehen könnte. **[3 BE]**

d) Gib für Insertionsort (Rangliste, n Einträge) die Zahl der Vergleiche im **besten** und im **schlechtesten** Fall an und nenne jeweils eine passende Eingabe für n = 5. Was passiert im besten Fall bei Verdopplung von n? **[3 BE]**

e) **Konstruiere** für die Rangliste mit 5 Einträgen eine **nicht** sortierte Eingabe, bei der `bubbleSort()` mit Abbruch schon nach **zwei** Durchläufen fertig ist. Gib die Zahl der Vergleiche an. **[2 BE]**

Aufgabe 3: Das Sortier-Duell

IDE ★★★

Jetzt wird es ernst: Welches Team sortiert mit den **wenigsten Vergleichen**? Ihr verbessert gemeinsam die Methode `duellSort()` in `MeinSortierer`. Zu Beginn ruft sie einfach euer `insertionSort` auf.

Die Duell-Regeln

1. Es tritt `MeinSortierer.duellSort()` an. Gestartet wird im Hauptprogramm mit `Duell.starten()` – Tempo vorher auf Maximum.
2. Vier Runden mit je **40 Balken**, für alle Teams exakt gleich: **zufall**, **fast sortiert**, **umgekehrt**, **viele gleiche**. Danach folgt eine **Kontrollrunde** mit 37 unbekannten Werten – sie zählt nicht, muss aber ebenfalls richtig sortiert werden.
3. **Wertung:** Summe der Vergleiche aus den vier Runden – **weniger ist besser**. Bei Gleichstand entscheidet die Summe der Vertauschungen.
4. **Erlaubt:** `Vis.laenge()`, `Vis.kleiner(i, j)`, `Vis.vertausche(i, j)`, eigene Variablen, Schleifen und Hilfsmethoden in MeinSortierer.
5. **Verboten** (macht das Ergebnis ungültig): `Vis.wert(i)` – liefert im Duell nur −1, sonst könnte man Werte ablesen, ohne einen Vergleich zu bezahlen · `Vis.starten`, `Vis.fertig` oder `Duell.starten` in duellSort · fest eingebaute Tauschfolgen für die bekannten Daten oder eine feste 40 statt `Vis.laenge()` · Änderungen an der Klasse Vis.
6. Am Ende erscheint die Zeile `DUELL-ERGEBNIS: Vergleiche = …, Vertauschungen = …, gueltig = ja`. Jedes Team meldet **ein** Ergebnis. Die Lehrkraft sammelt alle Ergebnisse an der Tafel bzw. am Beamer – das beste gültige Team gewinnt das Duell. (Das Duell ist ein Unterrichtsspiel; Punkte gibt es dafür nicht.)

a) Startet das Duell einmal mit dem unveränderten `duellSort` und notiert das Ergebnis als Ausgangswert.

b) Analysiert eure Tabelle aus Aufgabe 1 e): Wo verschwendet Insertionsort Vergleiche? Welches Wissen über den bereits sortierten Teil nutzt es nicht aus? Sammelt Ideen.

c) Verbessert `duellSort` Schritt für Schritt. Prüft jede Idee im Training auf **allen** Ausgangslagen (grün?) und dokumentiert:

Versuch	Idee in Stichworten	Summe Vergleiche	gültig?
1			
2			
3			
4			

Unser Duell-Ergebnis: Vergleiche _____ Vertauschungen _____ gültig ☐ ja ☐ nein

Denkanstoß: Messen statt glauben: Eine „Verbesserung“, die auf einer Ausgangslage spart, kann auf den anderen dreien mehr kosten. Nur die Summe zählt.

Achtung Abi: So wird Effizienz im Abitur geprüft

- **Zählen:** „Gib die Anzahl der Vergleiche für … an“ – mit Tracetabelle oder Summenformel.
- **Schlechtester Fall:** „Gib eine Eingabe an, für die … am meisten Vergleiche benötigt“ – und begründe, warum.
- **Verdopplung:** „Beurteile, wie sich die Laufzeit ändert, wenn sich die Anzahl der Einträge verdoppelt.“ Musterformulierung: *Verdoppelt man n, so vervierfacht sich die Zahl der Vergleiche ungefähr, da sie quadratisch von n abhängt: $2n(2n-1)/2 \approx 4 \cdot n(n-1)/2$.*
- Die **O-Notation** wird im niedersächsischen Abitur **nicht** verlangt. Argumentiere mit Zahlen.

Teil B – Binäre Suche (DS 16)

Aufgabe 4: Zahlenraten

PAPIER ★

Partnerarbeit, ohne Gerät: A denkt sich eine ganze Zahl von 1 bis 100. B rät; A antwortet nur „größer“, „kleiner“ oder „richtig“.

a) Spielt drei Runden mit wechselnden Rollen. Wie viele Versuche habt ihr jeweils gebraucht?

b) Beschreibt die beste Strategie für B. Wie viele Versuche braucht B damit **höchstens**? Begründet.

c) Wie viele Versuche wären es höchstens bei Zahlen von 1 bis 200? Und von 1 bis 1000?

Infoteil 2: Die binäre Suche

Die Strategie aus dem Zahlenraten heißt **binäre Suche** (binär: zweigeteilt). Sie funktioniert **nur auf sortierten Daten**: Ein Blick in die Mitte genügt, um eine ganze Hälfte auszuschließen.

- Die Reihung r ist **aufsteigend** sortiert. Gesucht ist x. Der noch mögliche Bereich reicht von Index **lo** bis Index **hi** (am Anfang die ganze Reihung).
- Die Mitte ist **mid = (lo + hi) / 2** – ganzzahlige Division, es wird abgerundet.
- r[mid] = x: gefunden, gib mid zurück. r[mid] < x: x kann nur rechts liegen → **lo = mid + 1**. r[mid] > x: x kann nur links liegen → **hi = mid − 1**.
- Wird der Bereich leer (**lo > hi**), kommt x nicht vor → −1.

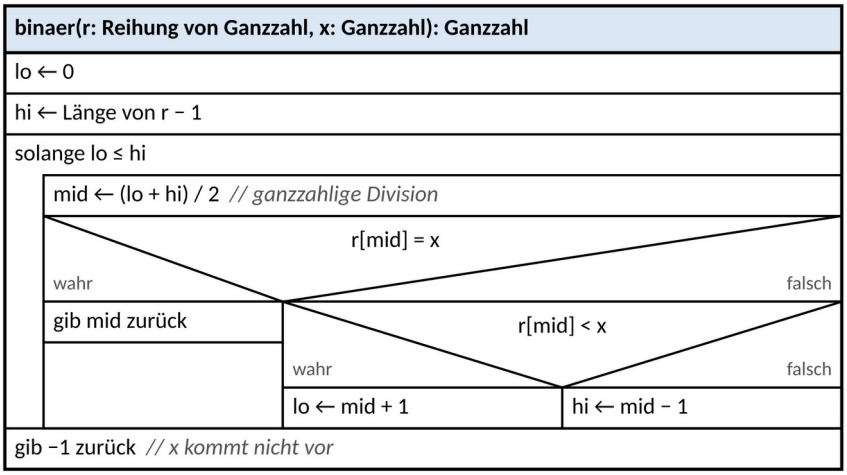


Abbildung 1: Binäre Suche (iterativ) nach EH § 5

Beispiel: Suche nach x = 990 in der Punktetabelle aus dem Hauptprogramm (n = 10):

Index	0	1	2	3	4	5	6	7	8	9
r[Index]	120	340	410	560	610	750	820	905	990	1200

Durchlauf	lo	hi	mid	r[mid]	Aktion
1	0	9	4	610	lo ← 5
2	5	9	7	905	lo ← 8
3	8	9	8	990	gib 8 zurück

Drei Durchläufe statt neun bei der linearen Suche. In IDE-Level 2.3 zählt die Klasse `Suche` genau das mit: `Suche.durchlaeufe` ist die Zahl der Schleifendurchläufe der letzten Suche.

Aufgabe 5: Trace: lo, hi, mid

PAPIER ★★

Gegeben ist dieselbe Punktetabelle (n = 10). Führe die binäre Suche nach Abbildung 1 von Hand aus. Trage in jedem Schleifendurchlauf eine Zeile ein, sobald mid berechnet ist (lo und hi zu Beginn des Durchlaufs); unter „Aktion“ steht, was danach geschieht.

Index	0	1	2	3	4	5	6	7	8	9
r[Index]	120	340	410	560	610	750	820	905	990	1200

a) Erstelle die Tracetabelle für die Suche nach **x = 820** und gib Rückgabewert und Zahl der Durchläufe an. **[3 BE]**

Durchlauf	lo	hi	mid	r[mid]	Aktion

Rückgabewert: _____ Durchläufe: _____

b) Erstelle die Tracetabelle für die Suche nach **x = 700** (kommt nicht vor). Gib an, mit welchen Werten von lo und hi die Schleife endet. **[3 BE]**

Durchlauf	lo	hi	mid	r[mid]	Aktion

Rückgabewert: _____ Durchläufe: _____ Ende mit lo = _____, hi = _____

c) Gib an, wie viele Elemente die **lineare** Suche jeweils prüft, bis sie 820 findet bzw. feststellt, dass 700 fehlt. **[1 BE]**

Kontrolle: Das Hauptprogramm im Gerüst führt genau diese Suchen aus – sobald dein IDE-Level 2.3 steht, vergleichst du dort.

Aufgabe 6: Wie oft höchstens?

PAPIER ★★★ ◆ nur LK (eA)

a) Bei n = 100 Elementen umfasst der Bereich lo … hi zu Beginn 100 Elemente. Gib an, wie viele Elemente nach jedem erfolglosen Durchlauf **höchstens** noch übrig sind, und begründe damit: Die binäre Suche braucht bei n = 100 **höchstens 7** Durchläufe. **[3 BE]**

nach Durchlauf	0	1	2	3	4	5	6	7
höchstens noch … Elemente	100							

b) Ergänze die Tabelle: höchste Zahl an Durchläufen der binären Suche. **[2 BE]**

n	1	2	4	8	16	100	200	1000	1 000 000
max. Durchläufe						7			

c) Begründe: Verdoppelt man n, braucht die binäre Suche höchstens **einen** Durchlauf mehr. **[2 BE]**

d) Vergleiche die lineare und die binäre Suche im schlechtesten Fall. **[2 BE]**

n	lineare Suche: geprüfte Elemente	binäre Suche: Durchläufe	Faktor
1 000			
1 000 000			

e) Eine **unsortierte** Punktetabelle hat 1 000 000 Einträge. Beurteile, ob es sich lohnt, sie zuerst mit Insertionsort zu sortieren, um danach binär zu suchen – wenn einmal gesucht wird und wenn 10 000-mal gesucht wird. Rechne mit dem schlechtesten Fall. **[3 BE]**

Aufgabe 7: Die Suchmaschine – IDE-Level 2.3

IDE ★ – ★★★

a) Öffne wieder `AQ_W2_L2_Rangliste_Geruest.txt`. Implementiere in der Klasse `Suche` die lineare Suche `linear(r, x)`: erstes Vorkommen, sonst −1. Setze `durchlaeufe` zu Beginn auf 0 und erhöhe es für jedes geprüfte Element.

b) Übersetze das Struktogramm aus Abbildung 1 in die Methode `binaer(r, x)`. Ergänze: `durchlaeufe` zu Beginn auf 0, in jedem Schleifendurchlauf + 1.

c) Starte `L23SucheTest`. Die Tests prüfen auch deine Trace-Ergebnisse (z. B. 4 Durchläufe für 90 in einer Reihung mit 9 Elementen) und die Höchstzahl 7 bei n = 100 bzw. 8 bei n = 200.

Stufe	Tests in L23SucheTest beginnen mit …
★ Basis	`basis_`
★★ Erweiterung	`erweiterung_`
★★★ Experte	`experte_`

d) Starte das Hauptprogramm und vergleiche die Ausgaben mit deinen Ergebnissen aus Aufgabe 5.

Achtung Abi: Die vier Fallen der binären Suche

- Sie funktioniert **nur auf sortierten Daten**. Ist die Reihung **absteigend** sortiert (wie die Rangliste!), drehen sich die Vergleiche um.
- Schleifenbedingung **lo ≤ hi** – mit lo < hi wird bei einem einzigen verbleibenden Element nicht mehr geprüft.
- **lo = mid + 1** und **hi = mid − 1** – wer nur lo = mid schreibt, erzeugt eine Endlosschleife (z. B. bei lo = 3, hi = 4).
- **gib −1 zurück** steht nach der Schleife, mid wird ganzzahlig berechnet.

Teil C – Speicherbedarf: Effizienz beurteilen (DS 16) ◆ nur LK (eA)

Infoteil 3: Speicherbedarf – die zweite Seite der Effizienz

- **Was zählt?** Der Speicherbedarf ist der **zusätzliche** Platz, den ein Algorithmus neben der Eingabe braucht: einzelne Hilfsvariablen und neu angelegte Reihungen (in Welt 3 kommt der Aufrufstapel bei der Rekursion dazu).
- **Konstant:** nur einzelne Variablen – z. B. Maximum, lineare und binäre Suche, aber auch Selectionsort, Insertionsort und Bubblesort, denn sie tauschen **innerhalb** der Reihung.
- **Linear:** eine Kopie oder Hilfsreihung der Länge n, z. B. `int[] neu = new int[anzahl];`
- **Quadratisch:** eine Reihung mit n Zeilen und n Spalten, z. B. `new int[n][n]` – doppeltes n, vierfacher Speicher.
- **Zeit gegen Platz:** Eine Strichliste `boolean[] gesehen` merkt sich, welche Werte schon vorkamen. Das spart Vergleiche, kostet aber Speicher – abhängig vom **Wertebereich**, nicht von n.
- **Beurteilen** heißt: Zahl der Operationen **und** Speicherbedarf abhängig von n abschätzen, die Verfahren vergleichen und für die konkrete Situation ein begründetes Urteil fällen.

Aufgabe 8: Wie viel Platz extra?

PAPIER ★★

a) Gib für jede Methode an, ob ihr zusätzlicher Speicherbedarf **konstant**, **linear** oder **quadratisch** von n abhängt, und begründe jeweils in einem Satz: `hoechstePunktzahl()` · `selectionSort()` · eine Methode `kopie()`, die `new int[anzahl]` anlegt und alle Punkte kopiert · eine Turniertabelle `new int[n][n]` für „jeder gegen jeden“. **[4 BE]**

b) Berechne, wie viele Felder die Turniertabelle bei n = 30 und bei n = 60 Spielern belegt, und gib an, um welchen Faktor der Speicherbedarf wächst, wenn sich n verdoppelt. **[2 BE]**

Aufgabe 9: Zwei Detektoren – beurteilen wie im Abitur

PAPIER ★★★

Ein Punktestand der Rangliste soll nur einmal vorkommen. **Verfahren A** vergleicht jeden Punktestand mit jedem späteren (zwei geschachtelte Schleifen wie bei `hatDoppelteNamen`). **Verfahren B** nutzt eine Strichliste `boolean[] gesehen = new boolean[1001]` – alle Punkte liegen zwischen 0 und 1000: Für jeden Punktestand p wird `gesehen[p]` abgefragt; ist es schon `true`, gibt es ein Doppel, sonst wird es auf `true` gesetzt.

a) Gib für n = 200 Punktestände ohne Doppel die Anzahl der Vergleiche von A und die Anzahl der Abfragen von B an. **[2 BE]**

b) Gib den zusätzlichen Speicherbedarf von A und von B an. **[2 BE]**

c) **Beurteile** die Effizienz beider Verfahren unter Abschätzung des Speicherbedarfs und der Zahl der Operationen. Berücksichtige auch den Fall, dass die Punktestände beliebige `int`-Werte sein dürfen. **[4 BE]**

Achtung Abi (nur Leistungskurs): Effizienz beurteilen

- **Beide Größen:** Zahl der Operationen und Speicherbedarf – jeweils abhängig von n (konstant, linear, quadratisch …).
- **Vergleich und Urteil:** Welches Verfahren ist wo besser, und wofür entscheidest du dich in der konkreten Situation (n groß oder klein, Wertebereich, Original soll unverändert bleiben …)? Ohne begründetes Urteil ist „beurteilen“ nicht erfüllt (P10).
- **Musterformulierung:** „B benötigt nur n Abfragen statt n(n − 1)/2 Vergleiche, dafür zusätzlich eine Reihung mit 1001 Feldern. Für große Ranglisten mit Punkten von 0 bis 1000 ist B daher effizienter; bei beliebigen int-Werten wäre die Strichliste zu groß – dann ist A vorzuziehen.“

Ausblick: O-Notation – nicht abiturrelevant

In Studium und Fachliteratur fasst man das Wachstum kurz zusammen: Man behält nur den am stärksten wachsenden Summanden und lässt konstante Faktoren weg.

Verfahren	Anzahl im schlechtesten Fall	kurz	gesprochen
lineare Suche	n	O(n)	linear
binäre Suche	etwa $\log_2 n$	O(log n)	logarithmisch
Selection-, Insertion-, Bubblesort	$n(n-1)/2 = ½\,n^2 - ½\,n$	$O(n^2)$	quadratisch

Ausblick auf Welt 3: Mergesort und Quicksort schaffen das Sortieren mit etwa $n \cdot \log_2 n$ Vergleichen – bei n = 1 000 000 sind das rund 20 Millionen statt 500 Milliarden. **Im niedersächsischen Abitur wird die O-Notation nicht verlangt** – dort argumentierst du mit Zählen, schlechtestem Fall und Verdopplung.

🎯 Im Lab üben: Level 2.3 bis 2.5 – Punkte gibt es dort automatisch

- **Level 2.3 „Das Sortier-Duell – Effizienz durch Zählen“** – zu Teil A (Aufgaben 0–3)
- **Level 2.4 „Halbieren statt Durchsuchen – binäre Suche“** – zu Teil B (Aufgaben 4–7)
- **Level 2.5 „Das Archiv – Speicherbedarf und Effizienz beurteilen“** – zu Teil C (Aufgaben 8–9, nur LK)

Im **Algo Quest Lab** (algoquest.frankjuchim.de) werden die Aufgaben automatisch geprüft, Punkte gibt es beim ersten Lösen. Der **Trainingsplatz** von Welt 2 stellt zusätzlich Zufallsaufgaben zur binären Suche. Danach wartet der **Online-Boss „Das Wettfischen am Drachensee“** (25 Minuten) – die beste Vorbereitung auf den Papier-Boss.

Für Schnelle: Die Einfügestelle (Stufe Experte)

Implementiere `einfuegeposition(r, x)` in der Klasse Suche: Sie liefert den **kleinsten Index i mit r[i] ≥ x** – also die Stelle, an der man x einfügen müsste, damit r sortiert bleibt. Ist x größer als alle Elemente, ist das r.length; bei einer leeren Reihung 0. Benutze die **binäre** Suche und zähle `durchlaeufe` mit. Die beiden Tests `experte_einfuegeposition…` gehören zur Stufe Experte.

- **Tipp:** Lass hi bei `r.length` starten (eine Stelle hinter dem Ende) und die Schleife laufen, solange lo < hi. Überlege: Wenn r[mid] ≥ x ist, kann mid selbst die Antwort sein – darf hi dann auf mid − 1 springen?
- Prüfe von Hand: Für r = {10, 20, 20, 30} muss sich für x = 20 die Position 1 ergeben, für x = 35 die Position 4.
- **Denkanstoß:** Hätte dir die binäre Suche im Sortier-Duell geholfen? Wo genau? Probiere es aus und vergleiche mit eurem Duell-Ergebnis.

Side-Quest für die Duell-Idee: freiwillig und ohne Punkte.